

Documentación Formal del Lenguaje Kimo

1. Introducción

Kimo es un lenguaje de programación diseñado con fines académicos para el estudio de compiladores, autómatas y lenguajes formales. El lenguaje incorpora estructuras básicas de programación imperativa, incluyendo:

- Declaración de variables
- Entrada y salida
- Expresiones aritméticas
- Condicionales
- Ciclos repetitivos
- Manejo básico de cadenas

El compilador de Kimo será implementado en Python y tendrá como objetivo generar código ensamblador de 16 bits compatible con arquitecturas x86.

2. Conjuntos Formales del Lenguaje

2.1 Alfabeto (Σ)

El alfabeto del lenguaje Kimo está definido por el siguiente conjunto de símbolos:

$\Sigma = \{A-Z, 0-9, +, -, *, /, \%, ^, =, >, <, !, ;, :, ,, ., (,), ", \, _, @, \#, \$, \&\}$

3. Símbolos Terminales (T)

3.1 Palabras Reservadas

Palabra Reservada	Descripción
-------------------	-------------

INICIO:	Marca el inicio del programa
INI_VAR	Inicio del bloque de variables
FIN_VAR	Fin del bloque de variables
LEER	Entrada de datos

IMPRIMIR	Salida de datos
SI	Inicio de condicional
ENTONCES	Bloque verdadero
SINO	Bloque alternativo
FIN	Fin de condicional
MIENTRAS	Inicio de ciclo while
HACER	Inicio del bloque iterativo
FINM	Fin de estructuras repetitivas
DESDE	Inicio de ciclo for
HASTA	Límite del ciclo for
ENT	Tipo entero
FLOAT	Tipo flotante
CAD	Tipo cadena

3.2 Operadores

Operadores Aritméticos

Operador	Descripción
+	Suma o concatenación
-	Resta
*	Multiplicación
/	División
%	Módulo
^	Potencia

Operadores Relacionales

Operador

Operador	Descripción
>	Mayor que
<	Menor que
!=	Diferente de
>=	Mayor o igual que
<=	Menor o igual que
==	Igual que

3.3 Delimitadores

Delimitador

Delimitador	Uso
;	Fin de instrucción
()	Agrupación
:	Declaración
,	Separador
"	Delimitador de cadenas

4. Expresiones Regulares Formales

4.1 Identificadores

<ID> ::= [A-Z0-9_-]+

Restricciones

- Pueden contener letras mayúsculas.
- Pueden contener números.
- Pueden contener " _ " y "-".

- Deben contener al menos un carácter.
- No pueden ser palabras reservadas.

Ejemplos válidos

A

TOTAL

NUM_1

A-1

123

4.2 Números Enteros

<ENTERO> ::= [+ -]?[0-9]+

Ejemplos válidos

5

-10

+20

4.3 Números Flotantes

<FLOTANTE> ::= [+ -]?[0-9]+\.[0-9]+

Ejemplos válidos

3.14

-0.5

+10.25

4.4 Cadenas

<CADENA> ::= "[A-Z0-9\s+\-*/%^=><!,:;._@#\$&]*"

Ejemplos válidos

"HOLA"

"COMPILADORES"

"123"

5. Tipos de Datos

Tipo	Descripción
ENT	Número entero
FLOAT	Número flotante
CAD	Cadena de caracteres

6. Estructura General del Programa

Sintaxis General

```
INICIO:

INI_VAR
  A : ENT;
  B : FLOAT;
  MENSAJE : CAD;
FIN_VAR

A = 10;
B = 3.14;

IMPRIMIR(A);

FIN
```

7. Proposiciones del Lenguaje

7.1 Proposiciones Simples

Proposición	Sintaxis	Ejemplo	Descripción
Declaración	<ID> : <TIPO>;	A : ENT;	Declara una variable
Asignación	<ID> = <EXPRESION>;	A = B + 5;	Asigna valores
Entrada	LEER(<ID>;	LEER(A);	Captura entrada
Salida	IMPRIMIR(<EXPRESION>;	IMPRIMIR(A);	Muestra datos
NOP	;	;	Instrucción vacía

7.2 Proposiciones Estructuradas

Proposición	Sintaxis
Condicional	SI <CONDICION> ENTONCES <BLOQUE> [SINO <BLOQUE>] FIN
While	MIENTRAS <CONDICION> HACER <BLOQUE> FINM
For	DESDE <ID> = <EXPRESION> HASTA <EXPRESION> HACER <BLOQUE> FINM

7.3 Ejemplos de Uso

Condicional

```
SI A > B ENTONCES
    IMPRIMIR("A ES MAYOR");
SINO
    IMPRIMIR("B ES MAYOR");
FIN
```

Ciclo While

```
MIENTRAS A > 0 HACER  
    IMPRIMIR(A);  
    A = A - 1;  
FINM
```

Ciclo For

```
DESDE I = 1 HASTA 5 HACER  
    IMPRIMIR(I);  
FINM
```

8. Gramática Libre de Contexto (GLC)

Símbolo Inicial

<PROGRAMA>

Producciones

<PROGRAMA> ::= INICIO: <BLOQUE_VARS> <STATEMENTS>

<BLOQUE_VARS> ::= INI_VAR <DECLARACIONES> FIN_VAR

| ϵ

<DECLARACIONES> ::= <DECL> <DECLARACIONES>

| <DECL>

<DECL> ::= <ID> : <TIPO>;

<TIPO> ::= ENT

| FLOAT

| CAD

<STATEMENTS> ::= <STATEMENT> <STATEMENTS>

| <STATEMENT>

<STATEMENT> ::= <ASIGNACION>

| <ENTRADA>

| <SALIDA>

| <CONDICIONAL>

| <WHILE>

| <FOR>

| ;

<ASIGNACION> ::= <ID> = <EXPRESION> ;

<EXP_NUM_SIMPLE> ::= <OPERANDO_NUM>

| <OPERANDO_NUM> <OP_ARIT> <OPERANDO_NUM>

<ENTRADA> ::= LEER (<ID>) ;

<SALIDA> ::= IMPRIMIR (<EXPRESION>) ;

<CONDICIONAL> ::= SI <CONDICION> ENTONCES <BLOQUE> <SINO OPC> FIN

<SINO_OPC> ::= SINO <BLOQUE>

| ε

<WHILE> ::= MIENTRAS <CONDICION> HACER <BLOQUE> FINM

<FOR> ::= DESDE <ID> = <EXPRESION> HASTA <EXPRESION> HACER <BLOQUE>
FINM

<BLOQUE> ::= <STATEMENTS>

<CONDICION> ::= <EXPRESION> <OP_REL> <EXPRESION>

<OP_REL> ::= >

| <

| ==

| !=

| >=

| <=

<EXPRESION> ::= <TERMINO> <OP_ARIT> <EXPRESION>

| <TERMINO>

<TERMINO> ::= <FACTOR> <OP_MULT> <TERMINO>

| <FACTOR>

<FACTOR> ::= (<EXPRESION>)

| <ID>

| <CONSTANTE>

<OP_ARIT> ::= + | - | * | / | % | ^

<CONSTANTE> ::= <ENTERO>

| <FLOTANTE>

| <CADENA>

<ENTERO> ::= [+ -]?[0-9]+

<FLOTANTE> ::= [+ -]?[0-9]+\.[0-9]+

<CADENA> ::= "[A-Z0-9\s+\-*/%^=><!,:;._@#\$&]*"

<ID> ::= [A-Z0-9_-]+

8.1 Expresiones Regulares Modulares

Átomos Léxicos Base

Símbolo	Expresión Regular
<ID>	[A-Z0-9_-]+
<ENTERO>	[+-]?[0-9]+
<FLOTANTE>	[+-]?[0-9]+\.[0-9]+
<CADENA>	"[A-Z0-9\s+\\-*/%^=><!;,:._@#\$&]*"
<TIPO>	ENT FLOAT CAD
<OP_REL>	>= <= == != > <
<OP_ARIT>	+ - * / % ^

<OPERANDO_NUM> ::= <ID> | <ENTERO>

<EXP_IMPRIMIBLE> ::= <ID> | <ENTERO> | <CADENA>

Producción	Expresión Regular Modular
<DECL>	^\s*(<ID>)\s*:\s*(<TIPO>)\s*;\$
<ENTRADA>	^\s*LEER\s*(\s*(<ID>)\s*)\s*;\$
<SALIDA>	^\s*IMPRIMIR\s*(\s*(<EXP_IMPRIMIBLE>)\s*)\s*;\$
<CONDICIONAL>	^\s*SI\s+(\s*(<OPERANDO_NUM>)\s*(<OP_REL>)\s*(<OPERANDO_NUM>)\s+ENTONCES\$
<WHILE>	^\s*MIENTRAS\s+(\s*(<OPERANDO_NUM>)\s*(<OP_REL>)\s*(<OPERANDO_NUM>)\s+HACER\$
<ASIGNACION>	^\s*(<ID>)\s*=\s*(.+?)\s*;\$
<EXP_NUM_SIMPLE>	^(<OPERANDO_NUM>)(?:\s*(<OP_ARIT>)\s*(<OPERANDO_NUM>))?\$

9. Símbolos No Terminales

Símbolo No Terminal	Descripción
<PROGRAMA>	Representa la estructura completa del programa
<BLOQUE_VARS>	Contiene las declaraciones de variables
<DECLARACIONES>	Lista de declaraciones
<DECL>	Declaración individual
<TIPO>	Tipo de dato
<STATEMENTS>	Secuencia de instrucciones
<STATEMENT>	Instrucción individual
<ASIGNACION>	Asignación de expresiones
<ENTRADA>	Instrucción de lectura
<SALIDA>	Instrucción de impresión
<CONDICIONAL>	Estructura condicional
<SINO_OPC>	Parte opcional del condicional
<WHILE>	Ciclo repetitivo while
<FOR>	Ciclo repetitivo for
<BLOQUE>	Grupo de instrucciones
<CONDICION>	Expresión lógica
<EXPRESION>	Expresión aritmética o concatenación
<TERMINO>	Parte multiplicativa
<FACTOR>	Unidad mínima evaluable
<OP_ARIT>	Operador aritmético
<CONSTANTE>	Valor literal
<ENTERO>	Número entero
<FLOTANTE>	Número decimal

<CADENA>	Literal de texto
<ID>	Identificador

10. Arboles

1. Declaración (A : ENT;)

```

      :
    /\
<ID> <TIPO>
    \
      ;

```

2. Asignación (A = 5;)

```

      =
    /\
<ID> <EXPRESION>
    \
      ;

```

3. Entrada de datos (LEER(A);)

```

      (
    /\
LEER )
    /\
<ID>;

```

4. Salida (IMPRIMIR(C);)

```

      (
    /\

```

```
IMPRIMIR  )  
          / \  
<EXP_IMPRIMIBLE> ;
```

11. Reglas Semánticas

Reglas Generales

1. Toda variable debe declararse antes de utilizarse.
2. No se permiten variables duplicadas.
3. El operador + funciona como:
 - suma para ENT y FLOAT
 - concatenación para CAD
4. No se permiten operaciones entre tipos incompatibles.
5. Los ciclos FOR incrementan automáticamente en 1.
6. Las condiciones deben evaluar expresiones compatibles.

```

# Producciones de Kimo
    # <DECL> ::= <ID> : <TIPO> ;
    self.regex_declaracion = re.compile(
        rf"^{self.ESP}({self.ID}){self.ESP}{self.DOSP}{self.ESP}({self.TIPO}){self.ESP}{self.PYC}$"
    )
    # <ENTRADA> ::= LEER ( <ID> ) ;
    self.regex_leer = re.compile(
        rf"^{self.ESP}{self.PR_LEER}{self.ESP}{self.PA}{self.ESP}({self.ID}){self.ESP}{self.PC}{self.ESP}{self.PYC}$"
    )
    #<SALIDA> ::= IMPRIMIR ( <EXPRESION> ) ;
    self.regex_imprimir = re.compile(
        rf"^{self.ESP}{self.PR_IMPRIMIR}{self.ESP}{self.PA}{self.ESP}({self.EXP_IMPRIMIBLE}){self.ESP}{self.PC}{self.ESP}{self.PYC}$"
    )
    # <CONDICIONAL> ::= SI <EXPRESION> <OP_REL> <EXPRESION> ENTONCES
    self.regex_si = re.compile(
        rf"^{self.ESP}{self.PR_SI}{self.ESP_OBL}"
        rf"({self.OPERANDO_NUM}){self.ESP}({self.OP_REL}){self.ESP}({self.OPERANDO_NUM})"
        rf"{self.ESP_OBL}{self.PR_ENTONCES}$"
    )
    # <WHILE> ::= MIENTRAS <EXPRESION> <OP_REL> <EXPRESION> HACER
    self.regex_mientras = re.compile(
        rf"^{self.ESP}{self.PR_MIENTRAS}{self.ESP_OBL}"
        rf"({self.OPERANDO_NUM}){self.ESP}({self.OP_REL}){self.ESP}({self.OPERANDO_NUM})"
        rf"{self.ESP_OBL}{self.PR_HACER}$"
    )
    # <FOR> ::= DESDE <ID> = <EXPRESION> HASTA <EXPRESION> HACER
    self.regex_for = re.compile(
        rf"^{self.ESP}{self.PR_DESDE}{self.ESP_OBL}"
        rf"({self.ID}){self.ESP}{self.IGUAL}{self.ESP}({self.EXP_NUM_FOR})"
        rf"{self.ESP_OBL}{self.PR_HASTA}{self.ESP_OBL}({self.EXP_NUM_FOR})"
        rf"{self.ESP_OBL}{self.PR_HACER}$"
    )
    # <ASIGNACION> ::= <ID> = <EXPRESION> ;
    self.regex_asignacion = re.compile(
        rf"^{self.ESP}({self.ID}){self.ESP}{self.IGUAL}{self.ESP}(.+){self.ESP}{self.PYC}$"
    )

```

```
self.regex_exp_num_simple = re.compile(  
    rf"^{self.OPERANDO_NUM})(?:{self.ESP}({self.OP_ARIT}){self.ESP}  
({self.OPERANDO_NUM}))? $"  
)
```